先进计算与数字工程研究中心

PATTERN RECOGNITION AND MACHINE LEARNING

Lecture 13 – Ensemble Learning I: Bagging, Random Forests, and the Bridge to Boosting

Chizhi Chris ZHANG

zhangchizhi@ciomp.ac.cn

Advanced Computing and Digital Technology Research Center

University of Chinese Academy of Sciences

Fall 2026

Three periods: average, randomize, interpret

Period	Question	We will calculate
1: 45 min	Why can averaging help?	Variance, correlation, bootstrap coverage
2: 45 min	How does a forest learn?	Votes, OOB predictions, split candidates
3: 45 min	How do we use and explain it?	Tuning, rare events, feature importance

Running example. Predict whether equipment will fail from sensor measurements. A deep tree fits training data well but changes sharply when a few observations change.

By the end. Explain both the benefit and the limits of combining trees; distinguish a reliable prediction from a persuasive feature ranking.

A tree can fit well and still be unstable

Idea

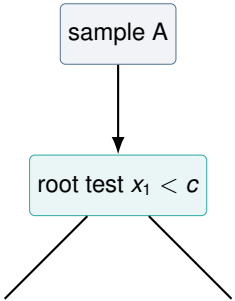
In a repair-cost regression illustration (costs in hundreds of currency units), a deep tree can change its first split when a small number of training cases change. This is sensitivity to the training sample, not merely a large training error.

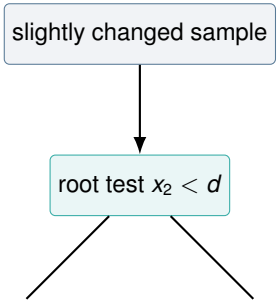
Work it out

Training sample	Train error	Validation error	Cost at x_0
A	0.00	0.22	18
B	0.00	0.24	30
C	0.00	0.21	12

What this tells us. All three trees fit their own training data. Their predictions vary widely at the same input. Averaging addresses this variation; it cannot create a missing relationship in the features.

A tree can fit well and still be unstable





A small data change can alter an early split and many downstream predictions.

Interpret prediction spread before choosing an ensemble

Understand

Compare repeated fits at the same repair-cost input. The predictions 18, 30, and 12 are expressed in hundreds of currency units.

Work through the example

$$\bar{f} = 20, \quad \frac{(18-20)^2 + (30-20)^2 + (12-20)^2}{3} = 56.$$

Interpretation. This is the population-style variance of three illustrative repeated-fit predictions, not the validation MSE of the whole model. The standard deviation is $\sqrt{56} \approx 7.48$. Averaging aims to reduce sample sensitivity; the true cost is still needed to assess bias.

Good learners also need different mistakes

	测试例 1	测试例 2	测试例 3
f_1	✓	✓	×
f_2	×	✓	✓
f_3	✓	×	✓
集成	✓	✓	✓

（a）集成提升性能

	测试例 1	测试例 2	测试例 3
f_1	✓	✓	×
f_2	✓	✓	×
f_3	✓	✓	×
集成	✓	✓	×

（b）集成不起作用

	测试例 1	测试例 2	测试例 3
f_1	✓	×	×
f_2	×	✓	×
f_3	×	×	✓
集成	×	×	×

（c）集成起负作用

Read the figure. Each column is one test case; each row is a classifier. Count correct votes in each column before inspecting the ensemble row.

Takeaway. Left: complementary errors allow a correct majority on every case. Middle: identical mistakes survive voting. Right: each weak member is correct only once, so the majority is wrong every time. Accuracy and diversity must be considered together.

Equal member accuracy can hide very different ensembles

Understand

On six binary cases, two collections of three classifiers can all have accuracy $4/6$, yet differ in where their mistakes occur.

Work through the example

Collection	Mistake sets of the three members	Majority errors
Complementary	A,B; C,D; E,F	0 of 6
Identical	A,B; A,B; A,B	2 of 6

Interpretation. In the first collection each case receives only one wrong vote; in the second A and B receive three wrong votes. Individual accuracy alone does not determine ensemble accuracy. This example isolates error overlap while keeping member strength fixed.

Three independent votes: calculate the benefit

Idea

For three binary classifiers, assume equal error probability $q = 0.30$ and independent errors on a test case. Majority voting fails when at least two classifiers fail.

Work it out

$$P(\text{wrong majority}) = \binom{3}{2} q^2(1-q) + q^3$$

$$= 3(0.3)^2(0.7) + (0.3)^3 = 0.216.$$

What this tells us. The ensemble error is 21.6%, below the 30% error of one member. Independence is a teaching assumption; classifiers trained on overlapping data usually have correlated errors.

Five independent votes: include all ways a majority can fail

Understand

With five independent binary classifiers and member error $q = 0.3$, majority voting fails with three, four, or five wrong votes.

Work through the example

$$P_5 = \sum_{k=3}^{5} \binom{5}{k} q^k (1-q)^{5-k}$$

$$= 0.13230 + 0.02835 + 0.00243 = 0.16308.$$

Interpretation. The five-member error is about 16.3%, compared with 21.6% for three independent votes and 30% for one member. The binomial calculation requires equal error probabilities and independent mistakes. It is a useful benchmark rather than a promise about overlapping bootstrap trees.

Practice: a weak advantage can still survive majority voting

Understand

Three independent members each have error $q = 0.4$. Predict the ensemble error before calculating, and explain why the answer cannot be obtained by simply averaging the three error rates.

Work through the example

$$P_3 = 3(0.4)^2(0.6) + (0.4)^3 = 0.288 + 0.064 = 0.352.$$

Interpretation. The ensemble improves from 40% to 35.2% error, less dramatically than the $q = 0.3$ example. If all mistakes were perfectly shared, error would remain 40%. If $q = 0.5$, independent majority error is also 0.5: voting does not create information from chance-level members.

Separate bias, variance, and observation noise

Idea

For squared-error regression at a fixed input, compare predictions over repeated training samples. The mean prediction may miss the true function; individual predictions may fluctuate around that mean.

Work it out

$$\mathbb{E}[(y - \hat{f}(x))^2] = \underbrace{(f(x) - \mathbb{E}\hat{f}(x))^2}_{\text{bias squared}} + \underbrace{\mathrm{Var}(\hat{f}(x))}_{\text{sample sensitivity}} + \underbrace{\sigma_\epsilon^2}_{\text{noise}}.$$

What this tells us. Example: bias $=$ 2, prediction variance $=$ 9, noise variance $=$ 4 gives expected squared error $4 + 9 + 4 = 17$. Averaging mainly targets the variance component; this decomposition does not directly equal a train–validation error gap.

A numerical bias–variance decomposition at one input

Understand

Assume a noiseless true cost $f(x) = 10$. Repeated training samples produce predictions $(6, 8, 10)$, whose mean is 8.

Work through the example

$$\text{Bias}^2 = (8 - 10)^2 = 4, \quad \text{Var} = \frac{4 + 0 + 4}{3} = \frac{8}{3}.$$

$$\mathbb{E}[(f - \hat{f})^2] = \frac{16 + 4 + 0}{3} = \frac{20}{3} = 4 + \frac{8}{3}.$$

Interpretation. The total error includes both a common offset and training-sample variation. Averaging these three predictions yields 8 and retains the offset from 10. This concrete decomposition uses population-style moments over the three equally likely training outcomes.

Independent averaging divides variance by the tree count

Idea

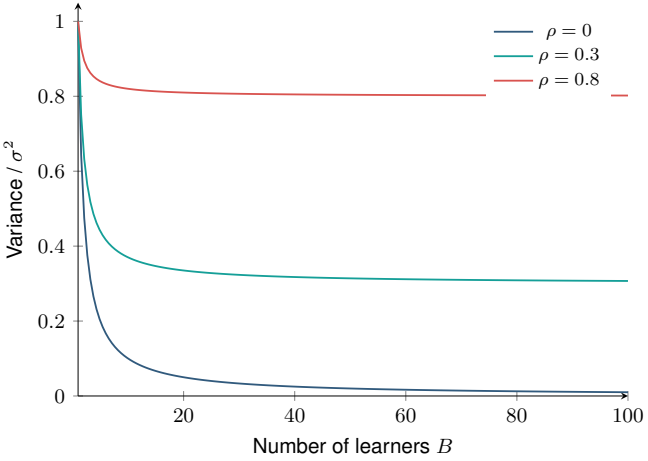
Let $f_1(x), \ldots, f_B(x)$ have the same variance σ^2 and zero pairwise covariance. We average their numeric predictions at a fixed input.

Work it out

$$\bar{f}(x) = \frac{1}{B}\sum_{b=1}^{B} f_b(x), \qquad \mathrm{Var}(\bar{f}) = \frac{1}{B^2}\sum_{b=1}^{B}\sigma^2 = \frac{\sigma^2}{B}.$$

What this tells us. If each prediction has variance 16, averaging four independent predictions gives variance 4 and standard deviation 2. The expected average is the average of member expectations, so a common systematic offset is preserved.

Independent averaging divides variance by the tree count



Unequal variances explain when unequal weights make sense

Understand

For two independent unbiased predictors, let their prediction variances be 1 and 9. Combine them as $af_1 + (1-a)f_2$.

Work through the example

$$V(a) = a^2 + 9(1-a)^2, \quad V'(a) = 2a - 18(1-a) = 0 \Rightarrow a = 0.9.$$

Interpretation. The resulting variance is $0.9^2 + 9(0.1)^2 = 0.90$, compared with 2.50 for equal weights. Giving more weight to the stable predictor helps under these assumptions. Bagging usually treats comparable peers equally; estimating reliable unequal weights requires data and can introduce additional overfitting.

Correlation leaves a floor under the variance

Idea

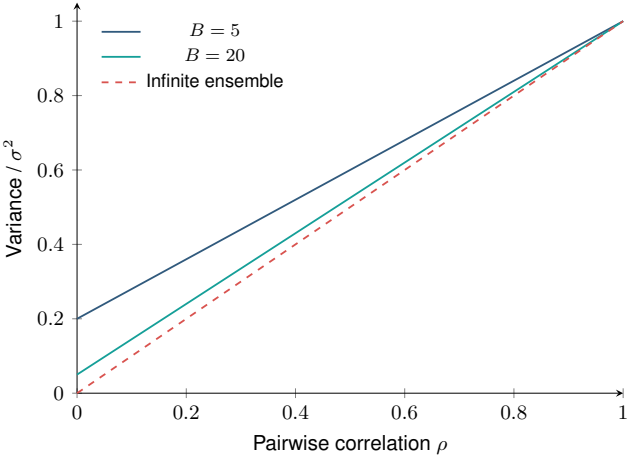
Now assume equal member variance σ^2 and equal pairwise correlation ρ. There are B variance terms and $B(B-1)$ covariance terms.

Work it out

$$\mathrm{Var}(\bar{f}) = \frac{B\sigma^2 + B(B-1)\rho\sigma^2}{B^2} = \sigma^2\left[\rho + \frac{1-\rho}{B}\right].$$

What this tells us. For $\sigma^2 = 16$, $\rho = 0.5$, and $B = 4$, variance is $16(0.5 + 0.5/4) = 10$, not 4. As B grows, it approaches $\rho\sigma^2 = 8$. More trees remove sampling variation around a correlated component.

Correlation leaves a floor under the variance



A shared component makes the variance floor concrete

Understand

Represent prediction error as $e_b = c + u_b$. All members share c, with variance 4; the independent member-specific terms u_b each have variance 12.

Work through the example

$$\mathrm{Var}(e_b) = 4 + 12 = 16, \quad \rho = 4/16 = 0.25.$$
$$\mathrm{Var}(\bar{e}) = 4 + 12/B \quad \Rightarrow \quad B = 4 : \ V = 7.$$

Interpretation. Averaging reduces the u_b component while leaving the same c in every prediction. This is a teaching model for common error, not a required structure for real forests. It reproduces the equal-correlation formula and explains why more members cannot remove all uncertainty.

Choose between more trees and less correlation

Idea

Compare two ways to improve an ensemble with member variance 16. Increasing B helps the independent part; reducing ρ changes the limiting variance.

Work it out

Correlation	10 trees	100 trees	Limit
$\rho = 0.5$	8.80	8.08	8.00
$\rho = 0.2$	4.48	3.33	3.20

What this tells us. Going from 10 to 100 highly correlated trees saves only 0.72 variance units. Reducing correlation at 10 trees saves 4.32 units, provided member strength is retained. Randomness is useful because of this tradeoff, not because random predictions are inherently good.

Measure overlap using errors, while respecting the interpretation

Understand

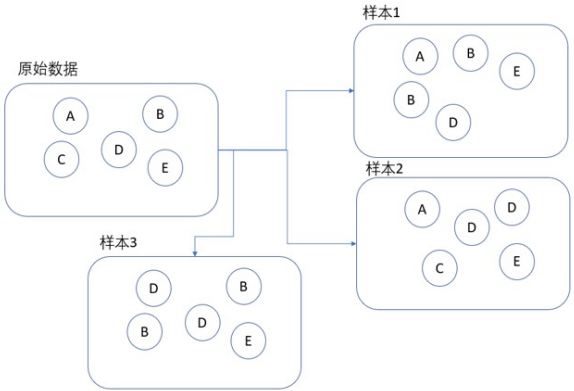
Three illustrative held-out error vectors are $e_1 = (-2, 0, 2)$, $e_2 = (-1, 0, 1)$, and $e_3 = (1, -2, 1)$. Every vector has mean zero.

Work through the example

$$e_1^\top e_2 = 4, \qquad e_1^\top e_3 = -2 + 0 + 2 = 0.$$

Interpretation. The first two vectors have correlation 1 despite different magnitudes; the first and third have correlation 0 on these cases. Such residual correlations diagnose shared mistakes on held-out inputs. They are not automatically the repeated-training, fixed-input correlation assumed in the earlier variance derivation.

Bootstrap samples reuse some rows and omit others



Read the figure

Start with A, B, C, D, E. Each sample contains five draws with replacement. In sample 1, B appears twice and C is absent. In sample 2, D appears twice and B is absent.

Takeaway

A bootstrap sample has the same number of draws as the original dataset, but fewer distinct observations. Duplicates change how strongly observations influence one learner; omitted rows can evaluate that learner.

Enumerate a tiny bootstrap instead of treating it as magic

Understand

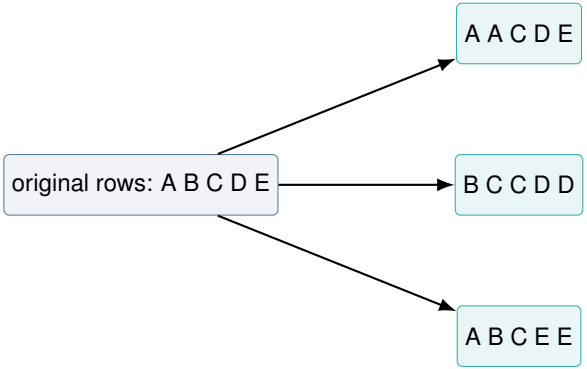
With original rows A and B, make two draws with replacement. The four equally likely ordered samples are AA, AB, BA, and BB.

Work through the example

Sample	Distinct rows	OOB rows
AA	1	B
AB or BA	2	None
BB	1	A

Interpretation. Calculate. Expected distinct count is $(1 + 2 + 2 + 1)/4 = 1.5$. A is omitted only by BB, with probability $1/4 = (1 - 1/2)^2$. Bootstrap preserves draw count, not unique-row count.

Enumerate a tiny bootstrap instead of treating it as magic



Duplicates act like observation multiplicities

Idea

For one bootstrap sample, store the number of times each original observation is drawn. Repeated rows can be represented by these counts when the learner supports equivalent sample weights.

Work it out

Original row	A	B	C	D	E
Draw count	1	2	0	1	1
Share of draws	0.2	0.4	0	0.2	0.2

What this tells us. The counts sum to 5, although only four rows are present. A different bootstrap changes the empirical distribution seen by the learner. The missing row C is OOB for this learner only; it may be included by another learner.

One row has a binomial draw count

Understand

For a fixed row in a five-row dataset, let K count how often it appears in five bootstrap draws. Each draw selects it with probability 0.2.

Work through the example

$$K \sim \text{Binomial}(5, 0.2), \quad \mathbb{E}K = 1, \quad \text{Var}\, K = 0.8.$$

$$P(K = 0) = 0.32768, \quad P(K = 1) = 5(0.2)(0.8)^4 = 0.40960.$$

Interpretation. The probability of appearing at least twice is $1 - 0.32768 - 0.40960 = 0.26272$. An expected count of one does not mean one appearance in every bootstrap sample. Omission and duplication both create meaningful variation in training influence.

Why about 36.8 percent of rows are out of bag

Idea

With N independent draws from N rows, a fixed row is missed once with probability $1 - 1/N$. To be OOB, it must be missed on every draw.

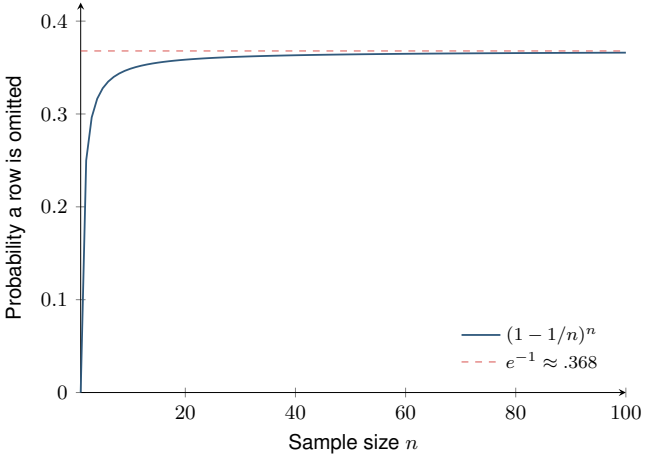
Work it out

$$P(\text{row is OOB}) = \left(1 - \frac{1}{N}\right)^N \longrightarrow e^{-1} \approx 0.368.$$

Dataset size	OOB probability	Expected distinct rows
$N = 5$	$(4/5)^5 = 0.32768$	$5(1 - 0.32768) = 3.36$
Large N	about 0.368	about $0.632N$

What this tells us. These are expected fractions, not fixed counts in each draw. Drawing N rows with replacement does not select all N distinct rows.

Why about 36.8 percent of rows are out of bag



Bootstrap draw count need not equal the original size

Understand

Draw m rows with replacement from N originals. A fixed row is OOB with probability $(1 - 1/N)^m$, approximately $e^{-m/N}$ for large N.

Work through the example

Draw ratio m/N	Approximate OOB fraction	Distinct-row fraction
0.5	0.607	0.393
1.0	0.368	0.632
2.0	0.135	0.865

Interpretation. Smaller samples give each learner less data and more omitted rows; larger samples reuse originals more often. Sample size changes both learner strength and diversity. The familiar 63.2% coverage specifically concerns $m = N$ and is not a universal bootstrap constant.

Practice: compute finite-sample coverage exactly

Understand

Four original rows are sampled four times with replacement. First compute the chance that a named row is omitted, then the expected number of distinct rows.

Work through the example

$$P(\text{OOB}) = (3/4)^4 = 81/256 \approx 0.3164.$$

$$\mathbb{E}[\text{distinct rows}] = 4(1 - 81/256) = 175/64 = 2.734375.$$

Interpretation. A particular sample contains an integer number of distinct rows, between one and four. The noninteger expectation summarizes repeated samples. Using 0.632 times four gives only a large-sample approximation; the exact finite calculation is preferable when N is small.

Activity: label the in-bag and OOB rows

Idea

Original data contain rows A through F. Two learners receive the bootstrap draws below. Work with a partner to identify omitted rows.

Work it out

$$D_1 = (A, A, C, D, D, F), \qquad D_2 = (B, C, C, E, F, F).$$

Learner	Distinct in-bag rows	OOB rows
1	A, C, D, F	B, E
2	B, C, E, F	A, D

What this tells us. C and F are used by both learners, so neither tree can give them an OOB prediction. Only two trees provide incomplete coverage. More independently drawn trees supply more eligible predictions.

Resample the independent unit when rows belong together

Understand

Five patients each contribute ten measurements. Resampling the 50 rows may split one patient between a learner's training rows and its OOB rows.

Work through the example

Resampling unit	One selected unit contains	What omission means
Individual row	One measurement	This row was not drawn
Patient	All ten patient measurements	This patient was not drawn

Interpretation. For patient-level bootstrap with five patients and five draws, a patient's omission probability is $(4/5)^5 = 0.32768$. If prediction concerns new patients, a row-level OOB estimate answers an easier question. Dependence determines the meaningful sampling unit.

Bagging trains peers and then combines predictions

ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Idea

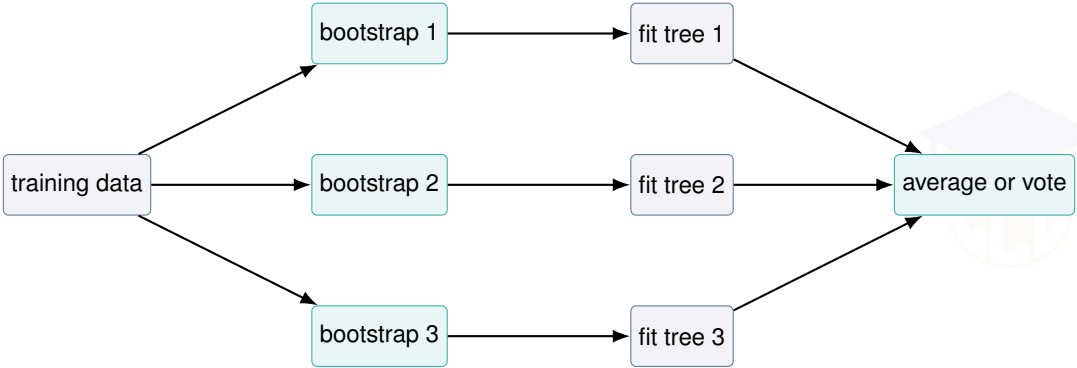
For each learner b, draw a bootstrap dataset D_b and fit the same learning algorithm. Every learner solves the original task; no learner is trained specifically to repair the previous one.

Work it out

1. Draw N rows with replacement from the training set.
2. Fit f_b on the drawn rows; repeat independently for $b = 1, \ldots, B$.
3. For a new input, average numeric outputs or aggregate classification predictions.

What this tells us. The learners can train in parallel. Data perturbation supplies diversity, and aggregation stabilizes predictions. This is bootstrap aggregation: Bagging.

Bagging trains peers and then combines predictions



A stable sample mean shows why bagging is not universally useful

Understand

Take targets 2 and 4 and use their sample mean as a very simple learner. The four two-draw bootstrap samples have means 2, 3, 3, and 4.

Work through the example

$$\mathbb{E}_{\text{bootstrap}}[\bar{y}^*] = (2 + 3 + 3 + 4)/4 = 3 = \bar{y}.$$

Interpretation. Averaging infinitely many bootstrap versions recreates the original sample mean in this example. Bagging does not automatically improve every estimator or create new observations. It is especially attractive when a learner responds unstably to sample perturbations, as deep trees often do.

Regression averages values, including their common bias

Idea

Three trees estimate a repair cost as 80, 110, and 95. Bagging gives equal weight to these numeric predictions.

Work it out

$$\hat{y} = \frac{80 + 110 + 95}{3} = 95.$$

What this tells us. The average is between the member predictions and may be less sensitive to a single unusual tree. If all trees underestimate a truly high repair cost, their average still underestimates it. Tree-based regression also tends not to extrapolate beyond the range of its training targets; averaging many trees does not remove that limitation.

Period-one exercise: more trees or more useful diversity?

Understand

Member variance is 16 and correlation is 0.6. Compare 10 trees with 100 trees. Then compare a 10-tree ensemble whose correlation has been reduced to 0.2 without weakening its members.

Work through the example

$$V_{10} = 16(0.6 + 0.4/10) = 10.24, \quad V_{100} = 9.664.$$
$$V_{10,\rho=0.2} = 16(0.2 + 0.8/10) = 4.48.$$

Interpretation. In this controlled comparison, reducing correlation helps more than multiplying the tree count. The conclusion assumes unchanged member variance and adequate strength. Explain which assumption may fail when candidate-feature sets become too small; this is the bridge to random forests.

Classification can combine labels or probabilities

Idea

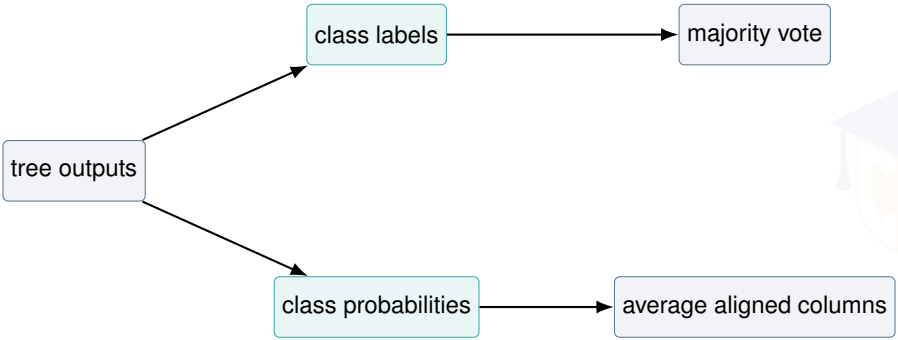
Hard voting counts predicted class labels. Soft voting averages class probabilities and then chooses a class or applies a decision threshold. Neither operation changes how the trees were trained.

Work it out

$$\hat{y}_{\mathrm{hard}} = \mathrm{mode}\{\hat{y}_b\}, \qquad \bar{p}_k = \frac{1}{B}\sum_b p_{b,k}, \qquad \hat{y}_{\mathrm{soft}} = \arg\max_k \bar{p}_k.$$

What this tells us. For multiclass prediction, align the class order before averaging probability columns. A random forest commonly averages leaf-derived class probabilities; this need not match majority voting on tree labels.

Classification can combine labels or probabilities



Multiclass probability averaging needs aligned class columns

Understand

Three trees predict probabilities for classes A, B, and C in the same column order. Read each row as one tree, and average each column.

Work through the example

Tree	A	B	C
1	0.60	0.30	0.10
2	0.20	0.50	0.30
3	0.10	0.20	0.70

Interpretation.

$$\bar{p} = (0.300, 0.333, 0.367).$$

Class C has the highest average probability. Hard labels give one vote per class, a tie. Soft aggregation retains probability strengths, but still depends on reliable estimates and correctly matched class labels.

The same three trees can produce different decisions

Idea

For one device, the three trees output failure probabilities 0.51, 0.51, and 0.01. Use a 0.5 decision threshold for this illustration.

Work it out

Tree	Failure probability	Hard class
1	0.51	Failure
2	0.51	Failure
3	0.01	Normal

$$\bar{p} = \frac{0.51 + 0.51 + 0.01}{3} = 0.343.$$

What this tells us. Hard voting predicts failure by two votes to one. Probability averaging predicts normal. The discrepancy reflects information discarded by hard labels; the choice should match probability quality and the intended decision rule.

Practice: soft voting can reverse the decision in either direction

Understand

Three trees assign failure probabilities 0.90, 0.49, and 0.49. Apply threshold 0.5 after each tree for hard voting, and after averaging for soft voting.

Work through the example

$$\text{Hard labels} = (F, N, N), \qquad \bar{p} = (0.90 + 0.49 + 0.49)/3 = 0.6267.$$

Interpretation. Hard voting predicts normal while soft voting predicts failure. The earlier example reversed the other way. Probability averaging is not a rule that always predicts fewer positives; it combines magnitudes. A cost-based threshold can then change the action without changing the averaged probability.

A tree probability comes from its reached leaf

ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Idea

A classification tree routes the input to one leaf. A simple probability estimate is the class fraction among the training observations reaching that leaf.

Work it out

$$p_b(\text{failure} \mid x) = \frac{n_{\text{failure, leaf}}}{n_{\text{leaf}}}.$$

What this tells us. A leaf with 1 failure among 2 cases estimates 0.5; a leaf with 50 failures among 100 cases also estimates 0.5. Their supporting evidence differs greatly. A forest averages leaf estimates, but averaging tiny leaves does not by itself establish trustworthy probabilities. Larger minimum leaf sizes can stabilize estimates.

A zero empirical leaf probability is not proof of zero risk

Understand

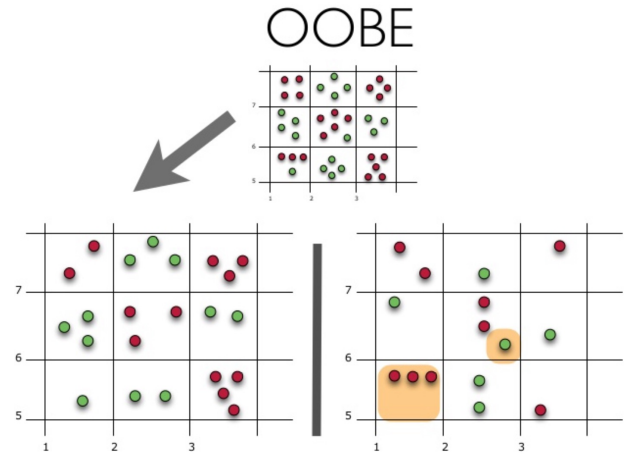
A leaf contains four normal cases and no observed failures. The empirical failure probability is 0. As an illustrative alternative estimator, add one pseudo-count to each class.

Work through the example

$$\hat{p}_{\text{empirical}} = 0/4 = 0, \qquad \hat{p}_{\text{smoothed}} = (0+1)/(4+2) = 1/6.$$

Interpretation. The pseudo-count estimate expresses uncertainty from a small sample; it is not the default rule of every random-forest implementation. Larger leaves and calibration are different strategies. An empirical zero describes observed leaf counts, not an impossible real-world event.

Out-of-bag evaluation uses the omitted observations



Read the figure

The top panel is the original data. A bootstrap learner is fitted using the selected observations in the lower-left panel; the lower-right observations were omitted from that learner.

Takeaway

Predict an omitted observation with this learner because it did not train on that observation. For a forest-level OOB prediction, average only eligible trees for each row. The eligible set changes from row to row.

OOB regression uses the same eligibility rule as classification

Understand

A training device has true repair cost 10. Trees 1 and 4 included this row; trees 2 and 3 omitted it. Their predicted costs are 25, 8, 12, and 9 respectively.

Work through the example

Tree	Eligible for this row?	Cost prediction
1	No	25
2	Yes	8
3	Yes	12
4	No	9

Interpretation.

$$\hat{y}_{\mathrm{OOB}} = (8 + 12)/2 = 10.$$

The OOB squared error is zero for this row. Averaging all trees gives 13.5, but that is a different prediction using in-sample members. Eligibility depends on sampling, not the task's output type.

Every row has its own eligible OOB trees

Idea

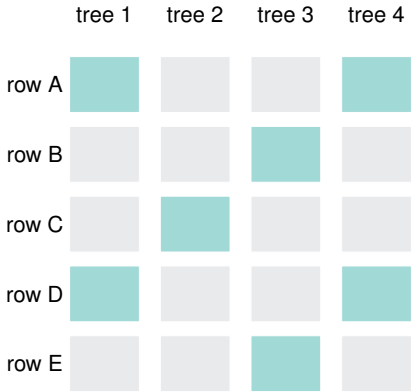
In this table, “in means a row was drawn for that tree; “OOB means it was not drawn. Select trees separately for each observation.

Work it out

Row	Tree 1	Tree 2	Tree 3	Eligible trees
A	in	OOB	OOB	2, 3
B	OOB	in	OOB	1, 3
C	in	in	OOB	3

What this tells us. Do not average all three trees when evaluating A: tree 1 has already seen A. Row C receives fewer eligible votes, so its OOB estimate is noisier. More trees improve coverage and reduce Monte Carlo variability of this internal estimate.

Every row has its own eligible OOB trees



Use only green cells: trees whose bootstrap samples omitted this row.

Score each OOB row once, rather than once per eligible tree

Understand

Three covered rows have true costs 10, 20, and 30, with OOB predictions 9, 18, and 33. They received different eligible-tree counts.

Work through the example

Row	Eligible trees	Squared error
A	12	$(10-9)^2 = 1$
B	40	$(20-18)^2 = 4$
C	5	$(30-33)^2 = 9$

Interpretation.

$$\mathrm{MSE}_{\mathrm{OOB}} = (1+4+9)/3 = 4.667, \qquad \mathrm{RMSE} = 2.160.$$

Different vote counts affect estimate stability, but the evaluation has three observations. Weighting by vote count would answer a different question.

Calculate one OOB probability and one OOB error

Idea

For A, tree 1 trained on A and gives probability 0.95; trees 2 and 3 omitted A and give 0.60 and 0.20. A is truly normal.

Work it out

$$\hat{p}_{\mathrm{OOB}}(A) = \frac{0.60 + 0.20}{2} = 0.40.$$

What this tells us. At threshold 0.5, A is correctly predicted normal. Averaging all trees would give 0.583 and the wrong class, but that is an in-sample forest prediction, not an OOB prediction. If OOB predictions are correct for 16 of 20 covered rows, OOB error is $4/20 = 0.20$.

OOB probabilities can be right but uncertain

Understand

Two covered cases have OOB failure probabilities 0.4 for a truly normal device and 0.7 for a truly failed device. Both decisions are correct at threshold 0.5.

Work through the example

$$L_A = -\log(1-0.4) = 0.5108, \quad L_B = -\log 0.7 = 0.3567.$$
$$\text{Mean log loss} = (0.5108 + 0.3567)/2 = 0.4338.$$

Interpretation. Classification error is zero for these two cases, but probability loss is not. The metric distinguishes a barely correct probability from a secure one. It does not by itself establish calibration; that concerns observed frequencies over comparable probability groups.

Tree count affects OOB coverage and stability

Idea

For large N, one tree omits a fixed row with probability about 0.368. With B independent bootstrap draws, its expected eligible-tree count is approximately $0.368B$.

Work it out

$$\mathbb{E}[B_{\mathrm{OOB}}] = 0.368B, \qquad P(B_{\mathrm{OOB}} = 0) \approx (0.632)^B.$$

What this tells us. At $B = 10$, about 3.68 trees are eligible and the no-vote probability is about 0.010. At $B = 100$, about 36.8 trees are eligible. More trees stabilize the aggregate and its OOB estimate, but do not repair biased sampling or a leaking feature.

Eligible-tree counts vary around their expectation

Understand

For large datasets, approximate a row's OOB count with $K \sim \text{Binomial}(B, q)$, where $q = 0.368$. Bootstrap draws for different trees are independent.

Work through the example

$$B = 100: \quad \mathbb{E}K = 36.8, \quad \text{Var}\, K = 100(0.368)(0.632) = 23.258.$$
$$\text{SD}\, K \approx 4.823.$$

Interpretation. Seeing 32 eligible trees for one row and 42 for another is unsurprising. The count distribution concerns how many trees may evaluate a row; it is different from the variance of the forest prediction. Coverage randomness should not be mistaken for a software error.

OOB and cross-validation answer related questions

Idea

OOB evaluates rows using trees that omitted those rows. Cross-validation refits the entire chosen learning procedure on separate training folds. They are useful estimates, but neither removes leakage automatically.

Work it out

Situation	Reasonable evaluation
Independent rows; fixed forest	OOB can be efficient
Tune preprocessing or select features	Cross-validate the whole procedure
Repeated devices or patients	Keep each entity in one group
Predict future observations	Respect temporal order

What this tells us. Repeatedly selecting parameters by OOB score can overfit that internal estimate. Keep an untouched final test set when estimating the performance of the selected procedure.

Precision changes when the evaluation prevalence changes

Understand

Assume a classifier has true-positive rate 0.8 and false-positive rate 0.1. Let π be failure prevalence in the evaluated population.

Work through the example

$$\text{Precision} = \frac{0.8\pi}{0.8\pi + 0.1(1-\pi)}.$$

Interpretation. At $\pi = 0.02$, precision is $0.016/(0.016 + 0.098) = 0.140$. At $\pi = 0.5$, it is $0.4/(0.4 + 0.05) = 0.889$. A balanced evaluation can therefore suggest far fewer false alarms than a rare-failure population. Compare OOB and holdout estimates using the intended class distribution and decision metric.

Row sampling alone may leave the same dominant split

Idea

Suppose temperature is much more predictive than every other sensor. Most bootstrap trees will choose temperature near the root, then make similar errors. Their samples differ, but their structures remain strongly related.

Work it out

- Bagging changes which rows and multiplicities a tree sees.
- A random forest also limits which features a node may consider.
- The best split is selected within that randomly chosen candidate set.

What this tells us. Feature randomness lowers similarity between trees when several useful alternatives exist. It can also weaken individual trees, so the candidate-set size controls a strength–correlation tradeoff.

When signal is sparse, feature randomness can weaken trees

Understand

Only one of 100 sensor features is informative. If a node selects 10 candidate features without replacement, the informative feature enters with probability $10/100 = 0.1$.

Work through the example

$$P(\text{useful feature available}) = m/d = 0.1.$$

Interpretation. The other 90% of candidate sets miss the only direct signal at this node. More diverse roots are not automatically better roots. Increasing candidate count can restore strength, while correlated alternatives can make smaller subsets safer. Availability is only the first step: the feature must also produce a good split on the node's cases.

Choose a new feature subset at each node

Idea

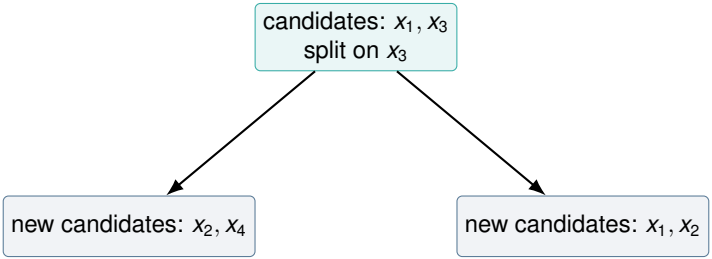
There are six features: temperature, vibration, current, humidity, age, and load. Let each node consider two randomly chosen features.

Work it out

Node	Candidates	Chosen split
Root	vibration, age	vibration < 2.1
Left child	temperature, load	temperature < 70
Right child	current, humidity	current < 8

What this tells us. Each node still searches thresholds for its own candidates. The tree may eventually use several features even though each split sees only two. A different tree receives different rows and different candidate subsets, producing a different partition of the input space.

Choose a new feature subset at each node



Resample candidate features at each node, then choose the best permitted split.

A feature may be used again farther down a tree

Understand

Feature subsampling is repeated at each node. The same temperature feature can be selected at the root and again in one of its children.

Work through the example

Path test	Resulting temperature region
Root: temperature < 70	Below 70
Left child: temperature < 40	Below 40 or between 40 and 70
Right child: temperature < 90	Between 70 and 90 or at least 90

Interpretation. A tree can refine one variable into multiple intervals. Candidate-feature count limits each search opportunity; it does not set the number of distinct variables or forbid reuse. Different paths can also use different feature combinations.

Still choose the best split among the allowed features

Idea

A node contains 6 failures and 4 normal cases, so its Gini impurity is $1 - 0.6^2 - 0.4^2 = 0.48$. Only vibration and age are allowed at this node.

Work it out

Candidate split	Child class counts (F,N)	Weighted child Gini	Reduction
vibration	(4,0) and (2,4)	0.267	0.213
age	(3,2) and (3,2)	0.480	0.000

What this tells us. Choose vibration: $0.48 - [4/10 \cdot 0 + 6/10 \cdot (1 - (2/6)^2 - (4/6)^2)] = 0.213$. A temperature split might be even better, but it is not a candidate at this node.

Practice: compare a third allowed split using weighted Gini

Understand

The parent contains 6 failures and 4 normal cases, with Gini 0.48. A third candidate produces left counts (3 failures, 1 normal) and right counts (3 failures, 3 normal).

Work through the example

$$G_L = 1 - (3/4)^2 - (1/4)^2 = 0.375, \quad G_R = 0.5.$$

$$G_{\text{children}} = 0.4(0.375) + 0.6(0.5) = 0.45; \quad \Delta G = 0.03.$$

Interpretation. Its gain is smaller than the vibration split's 0.213 in the preceding comparison. Evaluate every allowed candidate by the same weighted criterion. Splitting into two groups is not itself a gain; their class composition must become more informative.

How often does a useful feature enter the candidates?

Idea

With d available features and m candidates sampled without replacement, one particular feature is available with probability m/d. For s useful features, count candidate sets that miss all of them.

Work it out

$$P(\text{at least one useful feature}) = 1 - \frac{\binom{d-s}{m}}{\binom{d}{m}}.$$

What this tells us. If $d = 6$, $s = 2$, and $m = 2$, the probability is $1 - 6/15 = 0.60$. With $m = 3$, it is $1 - 4/20 = 0.80$. Larger candidate sets help trees find signal more often, but may also make their splits more similar.

Across independent roots, signal gets repeated opportunities

Understand

For 20 features and four candidates per root, a particular useful feature is available at any root with probability 0.2. Consider ten independently chosen root candidate sets.

Work through the example

$$P(\text{never available}) = (1 - 0.2)^{10} = 0.1074.$$
$$P(\text{available at least once}) = 1 - 0.1074 = 0.8926.$$

Interpretation. Adding trees increases opportunities to consider a feature, but availability does not guarantee selection or accurate prediction. The calculation concerns independent candidate draws at roots; it does not imply independent final tree errors or independent nodes within a learned tree.

Useful randomness balances strength and similarity

Idea

The table illustrates a controlled comparison using the same training split, tree count, leaf settings, and evaluation data. Only the candidate-feature fraction changes.

Work it out

Candidate fraction	Single-tree error	Error correlation	Forest error
All features	0.20	0.80	0.18
Half	0.23	0.45	0.13
Very small	0.36	0.15	0.24

What this tells us. The middle setting wins here: modestly weaker trees make less redundant mistakes. Extreme randomness loses too much signal. These illustrative numbers express a tradeoff, not a universal optimal fraction; choose the setting using an appropriate validation procedure.

Solve for a tree count instead of increasing it blindly

ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Understand

Under equal variance 16 and correlation 0.2, require ensemble variance at most 4. Solve the correlated-variance formula for B.

Work through the example

$$16(0.2 + 0.8/B) \leq 4 \Rightarrow 0.8/B \leq 0.05 \Rightarrow B \geq 16.$$

Interpretation. At $B = 16$, the idealized variance equals 4. A target below 3.2 is impossible without changing the variance or correlation, because $16 \times 0.2 = 3.2$ is the limiting floor. Real estimates may violate the simplified equal-correlation model, so this is a reasoning exercise rather than an exact tuning prescription.

A minimal forest fit connects to the mechanism

```
from sklearn.ensemble import RandomForestClassifier
forest = RandomForestClassifier(
    n_estimators=300, max_features="sqrt",
    min_samples_leaf=5, bootstrap=True,
    oob_score=True, random_state=13, n_jobs=-1)
forest.fit(X_train, y_train)
p_valid = forest.predict_proba(X_valid)[:, 1]
```

Read the arguments. 300 trees average many perturbed learners; square-root candidates add feature diversity; five observations per leaf restrain very small local estimates.

Check the meaning. OOB is an internal score; the validation data remains outside this fit. Class column 1 must correspond to the failure label.

Check integer counts and floating-point sample fractions

Understand

For 1,000 training rows, a sampling parameter may interpret an integer as a row count and a floating-point value as a fraction. Read the estimator's API definition.

Work through the example

Setting	Meaning in such an API	Draw count
1	One row	1
1.0	All rows as a fraction	1,000
0.5	Half of the rows	500

Interpretation. A one-character type difference can change the training task dramatically. Bagging and forest interfaces have version-specific names and constraints; check what `max_samples` means in the chosen implementation. Repeated draws still differ from distinct-row counts when bootstrap is enabled.

Extra Trees randomizes split thresholds too

Idea

A random forest usually searches useful thresholds within randomly selected features. Extra Trees proposes random thresholds and chooses a good split among those proposals. Common implementations may also use different bootstrap defaults.

Work it out

Method	Candidate features	Thresholds
Bagged trees	Often all	Search best
Random forest	Random subset	Search best within subset
Extra Trees	Random subset	Random proposals, then compare

What this tells us. On values $1, 2, 3, 8, 9$, an exact search considers midpoints such as 1.5, 2.5, 5.5, 8.5. Random proposals may miss 5.5. This saves search effort and adds diversity, but can increase bias.

Extra Trees example: random thresholds can miss the clean split

Understand

A regression node has feature values $(1, 2, 3, 8, 9)$ and targets $(0, 0, 0, 10, 10)$. Compare residual SSE after three thresholds.

Work through the example

Threshold	Leaf means	Total SSE
5.5	0 and 10	0
2.5	0 and $20/3$	$200/3 \approx 66.67$
8.5	2.5 and 10	75

Interpretation. Exact search finds the clean separation at 5.5. Random proposals at 2.5 or 8.5 retain mixed leaves, although multiple proposals can improve the chance of finding a useful partition. Threshold randomization trades split quality for speed and diversity; evaluate the full ensemble rather than one lucky proposal.

Parameters control different parts of the forest

Parameter	Mechanism	Question to ask
`n_estimators`	Number of averaged trees	Has the estimate stabilized?
`max_features`	Candidates per node	Is useful signal being missed?
`min_samples_leaf`	Local sample support	Are leaf estimates too noisy?
`max_depth`	Interaction and partition size	Is each tree too complicated?
`class_weight`	Class influence in splits	Which errors need more attention?

A practical start. Set enough trees for stable results; then compare a small grid of feature and leaf settings using the task metric. Change one mechanism at a time when learning how it affects the model.

Plan a small interpretable parameter comparison

Understand

Fix 200 trees and compare three candidate-feature settings with three minimum-leaf settings. Use the same five validation folds for every configuration.

Work through the example

Choice	Values	Number
Candidate features	2, 4, all	3
Minimum leaf size	1, 5, 20	3
Combinations	Every pair	9

Interpretation. Five-fold evaluation requires 45 fits. Compare the task metric and variation across folds, not the best training score. A small grid makes mechanisms understandable; expand it only when the results indicate useful directions. Tree count and parallel jobs should not be confused with feature candidates or leaf support.

Add trees until the prediction and score stabilize

Idea

Unlike making each tree deeper, adding trees mostly refines the average over the specified random-tree procedure. It increases training, storage, and prediction cost.

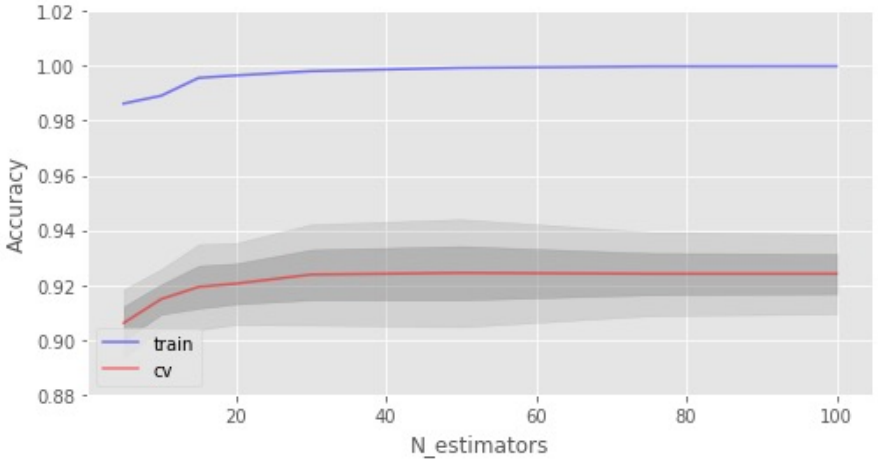
Work it out

Trees	OOB error	Validation log loss	Prediction time
50	0.145	0.410	1 unit
200	0.137	0.398	4 units
800	0.136	0.397	16 units

What this tells us. The last increase gives little benefit here. A sufficiently large forest need not be tuned as though tree count were tree depth, but resources and score stability still matter.

Do not confuse `n_estimators` with the number of features considered at a split.

Add trees until the prediction and score stabilize



More trees shrink only the independent standard-deviation part

Understand

In the shared-component model, common-error variance is 4 and independent member variance is 12. The standard deviation of the averaged independent part is $\sqrt{12/B}$.

Work through the example

Tree count	Independent SD	Total variance
100	0.346	4.12
400	0.173	4.03

Interpretation. Four times as many trees halves the independent standard deviation, while the total variance barely changes because the common floor remains. This distinguishes Monte Carlo precision from systematic model improvement. It also explains why tiny late-stage score gains may not justify four times the prediction work.

Leaf size changes probability support and fit

Idea

Compare two settings on the same data split. Accuracy measures decisions at a threshold; log loss also penalizes confident wrong probabilities.

Work it out

Minimum leaf size	Train accuracy	Validation accuracy	Validation log loss
1	1.00	0.89	0.47
10	0.95	0.90	0.35

What this tells us. Here larger leaves slightly reduce training fit but improve unseen probability quality. If the target depends on very fine local structure, excessively large leaves may underfit. Select the setting with validation evidence; use the metric that matches whether the output is a class decision, a ranking, or a probability.

A reliability table checks what a probability means

Understand

In three equal-size validation groups, compare typical predicted failure probability with the observed failure fraction. The values are illustrative.

Work through the example

Group size	Predicted probability	Observed fraction	Absolute gap
100	0.10	0.05	0.05
100	0.40	0.25	0.15
100	0.80	0.60	0.20

Interpretation. All three groups overestimate failure frequency. The equally weighted mean gap is $(0.05 + 0.15 + 0.20)/3 = 0.133$. This grouped calibration summary depends on binning and sample size; it complements log loss rather than replacing it. A good ranking can coexist with poorly calibrated probabilities.

Split according to the future prediction task

Idea

Suppose each device contributes 20 measurements. A random row split lets the same device appear in training and validation. The model may recognize device-specific patterns instead of learning to generalize to a new device.

Work it out

- New devices: split by device ID.
- Future measurements: split by time, using only earlier information.
- Independent observations: an ordinary stratified split may be suitable.
- Put fitted preprocessing and feature selection inside each training split.

What this tells us. A stable forest cannot compensate for an evaluation that asks an easier question than the intended application.

A grouped split can have different class rates naturally

Understand

Four devices contribute ten records each. Train on devices A and B and validate on C and D, rather than shuffling all 40 rows.

Work through the example

Device	Failure records	Normal records	Role
A	2	8	Train
B	0	10	Train
C	1	9	Validate
D	3	7	Validate

Interpretation. Training prevalence is $2/20 = 0.10$ and validation prevalence is $4/20 = 0.20$. A clean new-device boundary may expose real distribution differences. Report them and evaluate appropriately; forcing identical row-level class fractions can accidentally mix the same device across the boundary.

Rare failures make accuracy an incomplete answer

Idea

Out of 1,000 devices, 20 fail. Predicting every device as normal gives 98% accuracy and detects no failure. A forest threshold yields the following illustrative counts.

Work it out

Actual / predicted	Failure	Normal
Failure	15	5
Normal	35	945

$$\text{Recall} = 15/20 = 0.75, \quad \text{Precision} = 15/50 = 0.30.$$

What this tells us. This model finds 15 failures but creates 35 false alarms. Accuracy is 96%, below the all-normal baseline, while the decision may be far more useful. Judge it using the cost of missed failures and unnecessary inspections.

Translate the confusion matrix into expected operational cost

Understand

Return to 15 detected failures, five missed failures, and 35 false alarms among 1,000 devices. A miss costs 9 units and a false alarm costs 1.

Work through the example

$$C_{\text{forest}} = 5(9) + 35(1) = 80, \qquad C_{\text{all normal}} = 20(9) = 180.$$

Interpretation. The forest's lower accuracy can still save 100 total cost units in this illustration. Its mean cost is 0.08 per device versus 0.18 for the all-normal rule. The calculation assumes those costs and zero costs for correct actions; different inspection capacity or repair costs require a different decision model.

Class weights change the training impurity

Idea

If failures receive weight 5 and normal cases weight 1, a leaf with 2 failures and 8 normal cases has weighted class mass 10 versus 8.

Work it out

$$p_{\text{weighted}}(F) = \frac{5 \cdot 2}{5 \cdot 2 + 1 \cdot 8} = \frac{10}{18}, \qquad G = 1 - (10/18)^2 - (8/18)^2 \approx 0.494.$$

What this tells us. The unweighted failure fraction is only $2/10 = 0.20$. Weighted class masses change which splits are attractive. A weighted leaf proportion is not automatically the real-world failure probability; assess and, when needed, calibrate probabilities on data with the intended population distribution.

Balanced class weights equalize total class mass

Understand

For 1,000 observations with 20 failures and 980 normal cases, a common balanced-weight rule is $w_k = N/(Kn_k)$ with two classes.

Work through the example

$$w_F = 1000/(2 \cdot 20) = 25, \qquad w_N = 1000/(2 \cdot 980) = 0.5102.$$
$$20(25) = 500, \qquad 980(0.5102) \approx 500.$$

Interpretation. The weighted total mass of the two classes becomes equal. This is a training emphasis, not a claim that real failure prevalence is 50%. Balanced weights also do not encode every operational cost. Validate the resulting probabilities and decisions on the intended population.

A decision threshold follows the error costs

Idea

Let p be a calibrated failure probability. A false alarm costs $C_{FP} = 1$ unit, while a missed failure costs $C_{FN} = 9$ units. Correct decisions cost zero.

Work it out

$$\text{Cost(flag)} = (1-p)C_{FP}, \quad \text{Cost(normal)} = pC_{FN}.$$

$$\text{Flag when } p > \frac{C_{FP}}{C_{FP}+C_{FN}} = 0.10.$$

What this tells us. At $p = 0.20$, flagging costs 0.80 expected units, versus 1.80 for ignoring. The threshold changes the action after prediction; class weighting changes learning. The calculation depends on reliable probabilities and the specified costs.

A hard inspection budget changes the action rule

Understand

Six devices have calibrated failure probabilities $(0.9, 0.8, 0.7, 0.4, 0.3, 0.2)$. Only three inspections are available, with equal inspection costs and benefits per prevented failure.

Work through the example

$$\mathbb{E}[\text{failures found among top 3}] = 0.9 + 0.8 + 0.7 = 2.4.$$

Interpretation. Inspect the top three probabilities to maximize expected findings under these assumptions. Their expected false alarms total $3 - 2.4 = 0.6$. A fixed threshold of 0.1 would flag all six and violate the budget. Cost thresholds and resource-constrained ranking are related but different decision problems.

Impurity importance adds weighted split improvements

Idea

For a split at node t, measure how much weighted child impurity falls below the parent impurity. Weight that reduction by the fraction of training observations reaching the node.

Work it out

$$I_j \propto \sum_{t:\ \text{split feature}=j} \frac{n_t}{N}\left[G_t - \frac{n_L}{n_t}G_L - \frac{n_R}{n_t}G_R\right].$$

What this tells us. If 40% of observations reach a vibration split and its local Gini reduction is 0.20, its contribution is $0.4 \times 0.2 = 0.08$. Sum contributions over nodes and trees, then normalize. This measures how the fitted forest used features to reduce training impurity.

Normalize impurity contributions without overinterpreting them

Understand

Across the fitted forest, temperature contributes 0.12 weighted impurity reduction, vibration 0.06, and humidity 0.02. Normalize their sum to one.

Work through the example

$$I_T = 0.12/0.20 = 0.60, \quad I_V = 0.30, \quad I_H = 0.10.$$

Interpretation. The 60% figure is a share of this forest's accumulated split-improvement score. It does not say temperature causes 60% of failures or explains 60% of all uncertainty. Different splits, feature opportunities, and correlated alternatives can redistribute the normalized contributions.

Many possible thresholds can inflate impurity importance

Idea

A continuous measurement or an identifier with many distinct values offers many candidate splits. Even a useless feature can achieve an apparent training improvement by chance when enough thresholds are tried.

Work it out

Feature	Possible split opportunities	Training-importance concern
Binary switch	Few	Limited threshold search
Continuous sensor	Many	More chances for random gain
Device ID	Often very many	May memorize identity

What this tells us. A high impurity importance can reflect search opportunities or overfitting. Check whether the feature is legitimate at prediction time and whether its disruption harms performance on unseen, appropriately split data.

A high training importance can disappear on new devices

Understand

Consider this illustrative diagnosis for a device-ID feature. The forest uses it frequently in training, but disrupting it on new-device validation has almost no performance effect.

Work through the example

Feature	Normalized impurity score	Held-out permutation loss increase
Device ID	0.45	0.001
Temperature	0.30	0.16

Interpretation. The disagreement prompts investigation of memorization and split opportunities; it is not resolved by choosing the prettier ranking. Evaluate whether ID is meaningful and available for the intended task. Training split usage and unseen-data dependence answer different questions.

Permutation importance asks what prediction loses

Idea

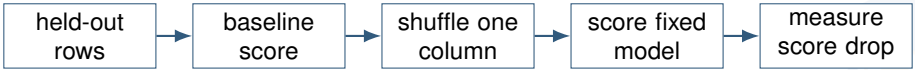
First score the fitted forest on validation data. Then shuffle one feature column across validation rows, keeping labels and other columns fixed. Rescore without retraining.

Work it out

Validation input	Log loss	Increase over baseline
Unchanged	0.32	0.00
Shuffle temperature	0.48	0.16
Shuffle vibration	0.39	0.07
Shuffle humidity	0.321	0.001

What this tells us. For a loss, importance is the increase after shuffling; for a higher-is-better score, it is the decrease. Repeat shuffles to assess variation. The result describes this model, this dataset, and this metric.

Permutation importance asks what prediction loses



Repeat permutations to see their conditional variability

Understand

Three shuffles of temperature increase validation log loss by 0.14, 0.16, and 0.18. Keep the fitted model and validation rows fixed.

Work through the example

$$\bar{\Delta} = 0.16, \qquad s = \sqrt{\frac{(-0.02)^2 + 0^2 + 0.02^2}{2}} = 0.02.$$

Interpretation. These repetitions measure variability due to the shuffle operation. The standard error of their mean is approximately $0.02/\sqrt{3} = 0.0115$, conditional on this model and validation dataset. It is not the full uncertainty from collecting new devices or refitting the forest; those require additional resampling or validation designs.

Correlated features can substitute for each other

Idea

Temperature sensor A and sensor B measure nearly the same quantity. The forest may substitute one for the other, so shuffling a single sensor can produce only a small loss increase.

Work it out

Disrupted feature group	Validation log-loss increase
Sensor A only	0.01
Sensor B only	0.02
A and B together	0.18

What this tells us. The group matters although each individual score is small. Joint shuffling preserves their paired values while breaking their connection to the row. Correlated predictors also make single-feature permutations less realistic; interpretation may require grouped or conditional analyses and domain knowledge.

Removing a feature and shuffling it ask different questions

Understand

Two temperature sensors are highly redundant. Compare disrupting one feature in the fitted model with retraining a model that never receives it. The loss values are illustrative.

Work through the example

Procedure	Validation log loss	What changes
Original model	0.35	Nothing
Shuffle sensor A	0.36	Input to the fixed model
Refit without A	0.35	Model can learn to use B
Refit without A and B	0.54	Temperature group is unavailable

Interpretation. Permutation diagnoses the existing model's dependence. Feature-removal experiments evaluate a new learning procedure. Retraining may exploit substitutes, so neither score directly answers the other question.

Feature selection must be fitted inside the split

```
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier
model = make_pipeline(
    SimpleImputer(strategy="median"),
    RandomForestClassifier(n_estimators=300,
        max_features="sqrt", min_samples_leaf=5,
        random_state=13))
model.fit(X_train, y_train)
```

Keep the order. Fit imputation and the forest on training data; evaluate on validation data. If importance is used to select columns, repeat that selection inside each training fold.

Why. Otherwise validation labels can influence which features are kept, making later validation scores optimistic.

Feature-selection folds must keep their own decision boundary

Understand

A three-fold feature-selection experiment may choose different sensor subsets in different training folds. Selection uses training labels, then evaluation uses that fold's held-out rows.

Work through the example

Fold	Select on training part	Evaluate on held-out part
1	temperature, vibration	Fold 1 only
2	temperature, load	Fold 2 only
3	vibration, age	Fold 3 only

Interpretation. Do not select one subset using all labels before cross-validation. Selection instability can reveal limited evidence or interchangeable predictors. Report the performance of the entire selection-and-fit procedure; a final subset can then be chosen using the training data after evaluation is complete.

Case study: use the evidence to choose a forest

Idea

All rows below use device-grouped validation and the same preprocessing. The figures are illustrative. The goal is useful failure probabilities with limited inspection capacity.

Work it out

Model	Recall	Precision	Log loss
Deep single tree	0.60	0.23	0.61
300 bagged trees	0.70	0.28	0.42
300-tree random forest	0.75	0.30	0.35

What this tells us. First, averaging improves the unstable tree. Feature randomness provides a further gain here. Before increasing trees again, inspect leaf size, probability reliability, and threshold effects. None of these scores proves the sensors cause failure.

Average residuals case by case, then compare MSE

Understand

Three regression learners have held-out error vectors $(1, 3, -2)$, $(-1, 1, 2)$, and $(0, 2, 0)$. Errors are prediction minus target.

Work through the example

$$\bar{e} = (0, 2, 0), \qquad \mathrm{MSE}(\bar{e}) = 4/3.$$

Interpretation. Individual MSE values are $14/3$, $6/3$, and $4/3$. Averaging reduces errors where positive and negative deviations cancel, but all members remain high on the second case. The ensemble matches the best member here rather than beating it. Diagnose the remaining shared pattern before deciding to add more peers.

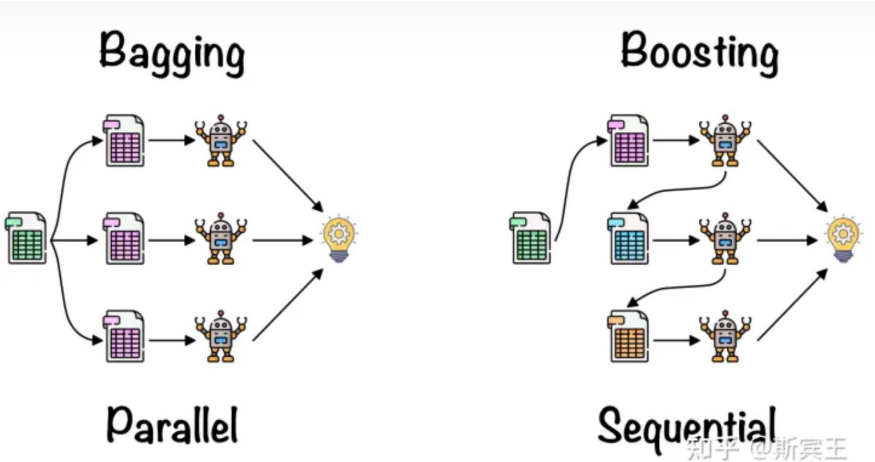
Bagging peers average; boosting successors correct

Idea. Bagging fits every learner to the original target on a perturbed sample. Boosting fits a sequence in which the next learner receives information about the current ensemble error.

	Bagging / random forest	Boosting
Relationship	Parallel peers	Sequential corrections
Learning signal	Original targets	Weighted mistakes or gradients
Combination	Usually equal average	Add weighted corrections

Tiny preview. If current regression predictions are $(8, 12)$ and targets are $(10, 9)$, the next boosting learner is asked to approximate residuals $(2, -3)$. It adds a correction rather than another independent estimate of $(10, 9)$.

Bagging peers average; boosting successors correct



Use the residual bridge to calculate one boosting step

Understand

Two true costs are $(10, 9)$ and current predictions $(8, 12)$. A new learner proposes the exact residual correction $(2, -3)$; apply learning rate 0.5.

Work through the example

$$F_{\text{new}} = (8, 12) + 0.5(2, -3) = (9, 10.5).$$
$$\text{SSE}_{\text{old}} = 2^2 + (-3)^2 = 13, \quad \text{SSE}_{\text{new}} = 1^2 + (-1.5)^2 = 3.25.$$

Interpretation. The second learner has a new target: the remaining error. This is not another independent estimate of the original costs. Real trees only approximate a residual function, so the exact correction here is an idealized teaching case.

Explain the whole forest in three connected steps

Question	Answer to reconstruct
Why average?	$\mathrm{Var}(\bar{f}) = \sigma^2[\rho + (1-\rho)/B]$
Why randomize?	Row and feature choices reduce redundant errors
How evaluate?	Use eligible OOB trees or an appropriate held-out procedure
How interpret?	State the metric and distinguish training from held-out importance

Exit exercise. A forest has tiny leaves, high training accuracy, poor validation log loss, and correlated temperature sensors. Propose one change for probabilities and one change for interpretation.

Expected reasoning. Compare larger leaves on validation data; estimate grouped permutation importance for the temperature sensors. More trees alone does not answer either problem.

Complete the forest reasoning with one integrated exercise

Understand

Use five-row bootstrap sampling, two eligible OOB probabilities 0.2 and 0.8, and a failure-action threshold 0.1. Member prediction variance is 16, correlation 0.5, and tree count 10.

Work through the example

$$P(\mathrm{OOB}) = (4/5)^5 = 0.32768, \quad p_{\mathrm{OOB}} = 0.5, \quad V = 16(0.5 + 0.5/10) = 8.8.$$

Interpretation. The action is to flag because 0.5 exceeds 0.1. OOB probability, decision threshold, and prediction variance describe different objects. Finish by explaining why a high temperature-importance score is model dependence rather than proof of a causal effect.